```
(venv) RiffMates$ python manage.py test
➥ bands.tests.TestBands.test_musician_view
```

Man, is that a lot of typing. The main reason I wrote `WRunner` was to provide a shortcut for calling a single test. `WRunner` treats an equal sign (`=`) prefix as shortcut and looks for any test in your suite that matches what comes after the sign. The previous test could be found with

```
(venv) RiffMates$ python manage.py test =music
```

It doesn't stop at the first match, so if you have several tests with the word *music* in their names, they'd all get invoked. As I'm already doing self-promotion, here is one more: `context_temp` (https://github.com/cltrudeau/context_temp). This library has two context managers, one that creates a temporary directory and another that creates a temporary file. The temp file or directory gets created when the context block gets entered and removed when it exits. If you need a temporary file or directory and don't want to worry about cleaning them up, this context manager tool could be for you.

8.8 Measuring tests and code quality

How do you know if you've tested everything? You can never be 100% sure, but there are things you can measure. Coverage (https://pypi.org/project/coverage/) is a third-party library you use when running your tests to track what project code got executed. After your tests complete, you get a summary showing the percentage of your code that ran under the test.

Measuring coverage gives you a sense of how much of your code has been tested, but it isn't perfect. Just because a line executed doesn't mean all possible conditions for that line got checked. Also, your code might have flaws that your tests don't trigger. In certain cases, failing to check for `None` may cause your code to crash, but if you don't have a test that checks it, your 100% coverage statement is a fake safety blanket. It isn't a silver bullet, but you can't improve what you can't measure, so it is still a good idea.

In addition to checking your code's functionality, you may want to check its quality. This is another deep subject, complete with fun phrases like *code smell*. There are dozens of tools out there that give feedback on your code. I typically have a shell script in my projects that runs the tests, measures the coverage, and runs a quality measure. If anything complains, I try to fix it immediately.

The following are some code quality tools you might be interested:

- *Linters*—These check for logical and stylistic problems. Popular Python linters are PyFlakes (https://github.com/PyCQA/pyflakes), Pylint (https://pypi.org/project/pylint/), ruff (https://pypi.org/project/ruff/), and MyPy for type enforcement (http://mypy-lang.org/).
- *Reformatting tools*—These make your code compliant with common style recommendations. Popular Python reformatting tools are Black (https://github.com/ambv/black), ruff (https://pypi.org/project/ruff/), and isort for import sorting (https://github.com/timothycrosley/isort).